26-Week Embedded Systems Curriculum

21 Oct 2026 · @Sam

This plan takes you from first circuits to three portfolio projects in 26 weeks (26 Oct 2026 – 25 Apr 2027), and every one of the 36 Required items on the Embedded Systems Engineering Roadmap has a home in it. It stays broad on purpose: you choose a direction at Gate 4, not before.

How to use this printout

You have about 20 hours a week for 26 weeks, and the plan is sized to that.

- **Dates:** Week 1 starts Monday 26 October 2026; week 26 ends Sunday 25 April 2027. Tick anything you have already done straight away.
- **Time:** about 20 hours a week: 8 building, 7 learning, 3 documenting, 2 spare for debugging. If you reliably have more than 20, spend the surplus on the stretch menu (section 12), not on stretching the core weeks.
- **Two threads:** beside each week's topic, a C thread (2 hours a week, weeks 2–14) and a math thread (1 hour a week, weeks 1–20) run inside the learning hours.
- **Ticking rule:** tick a box only when you can do the thing without a tutorial open.
- **Cold question:** each week ends with one question you answer aloud without notes. The week isn't done until you can.
- **Three projects:** renewable energy (weeks 11–14), marine conservation (weeks 15–20), practical IoT (weeks 21–25). Each ends with a GitHub repo, a system diagram and a one-page brief.
- **Firmware Lab:** two short blocks (weeks 13–14 and 19–20) on an STM32 Nucleo board. They run while Project 1's outdoor test and Project 2's deployment are soaking, so they use waiting time.
- **Gates:** four review points ask whether the work still energises you. Gates 1–3 also include two 45-minute "landscape tasters": short videos on areas the plan doesn't cover, so you can sample without committing.
- **Christmas:** weeks 9–10 (21 Dec – 3 Jan) are software-only, so they flex around the holidays.
- **Roadmap lines:** each week ends with a "Covers" line naming the roadmap items it touches. Section 11 lists them all with depth levels.

How the plan is grounded in the roadmap

Every Required item on the roadmap is placed in a specific week or project, so you can check the plan against the source at any time.

- **Source:** Embedded Systems Engineering Roadmap by Meysam Parvizi, v2.0.3 (updated 11 Oct 2026), CC BY-SA 4.0. It is a community-maintained open-source map, not a peer-reviewed one. It is widely used (about 13k GitHub stars) and its definition of an embedded system cites ISO/IEC/IEEE 24765. It says priorities vary by role and that nobody should try to study everything.
- **Required (36 items):** all in the core plan.
- **Recommended (27 items):** core when it takes about 2 hours or fits a project you are already building; otherwise stretch or a taster.
- **Optional:** tasters or stretch, chosen for breadth rather than depth.
- **Staying broad:** the roadmap spans software, hardware and soft skills, so the plan does too. Firmware, field systems and conservation technology stay open until Gate 4, when your tagged postings and your energy scores point one way or another.
- **Workload:** this adds about 95 hours to your original draft (C thread 26, math 20, Firmware Lab 22, testing, simulation and tools about 14, digital design, SPI and oscilloscope about 8, tasters and KiCad about 6). Dropping the documenting hour frees 26 hours of learning time and the Lab uses waiting time, so the plan assumes you reliably have 20 hours.

Three cross-checks keep the grounding honest:

- [ ] Week 8: skim the course list of each Camosun program in your tracker; note any topic missing from the skills map
- [ ] Weeks 1–26: tag every saved job posting with skills-map items (section 3)
- [ ] Week 26: compare the roadmap, the Camosun course lists and your posting tags; note where they disagree

Target roles and what the six months must prove

The six months must show you can build the hardware side credibly, alongside software, architecture and UX judgement. These roles are hypotheses to test against real postings, not confirmed openings.

Role	Why your background fits	How to test it
IoT solutions engineer or architect	Direct reuse of architecture and integration experience	Project 3 architecture brief; read 5 postings

Role	Why your background fits	How to test it
Embedded software or firmware developer (junior)	Software discipline plus hands-on C, debugging and RTOS work	Firmware Lab; read 5 postings; note C and RTOS requirements
Environmental sensor systems technician	Data pipelines plus hands-on fault-finding	Projects 1 and 2; note required certificates
Marine field instrumentation technician	Documentation, diagnosis, working in a crew	Marine safety tickets; ask AXYS or ONC about entry routes
Electronics or hardware technician (test, repair, assembly)	Soldering, measurement and fault-finding from Phase A	Ask the makerspace and a local electronics shop about entry routes
Conservation technology specialist	Solution design with user empathy	Talk to one local conservation group about a measurement they need
Assistive technology designer	UX plus physical prototyping	Project 3 variant with an accessible interface

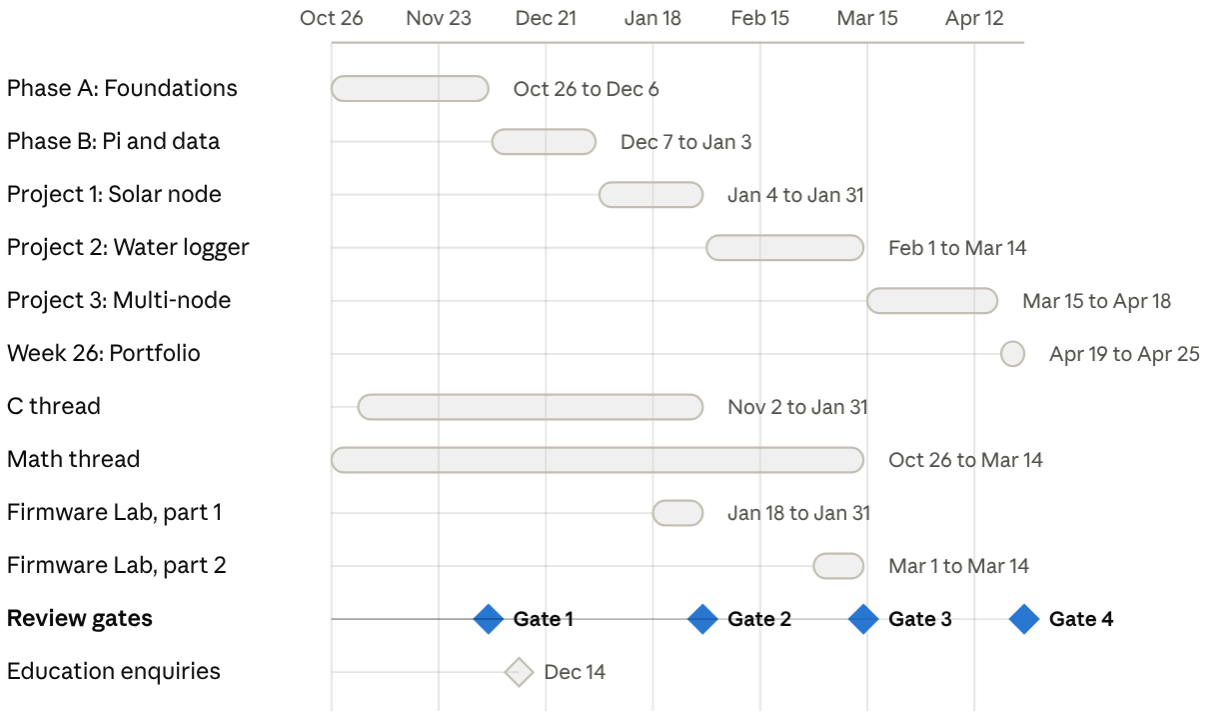
Evidence to collect across the six months:

- [ ] Save 15 job postings across these roles; tag required skills, certificates, pay range and the skills-map items each names
- [ ] Mark each posting firmware-heavy (C, RTOS, debugging) or field-and-systems-heavy (sensors, installation, protocols, certificates)
- [ ] Check that typical entry pay in at least 3 postings is workable for you
- [ ] Hold 3 informal conversations with people in these roles (ONC, AXYS, MarineLabs, a conservation group)
- [ ] After each gate, name the two roles that now look most plausible
- [ ] Any parked skill that appears in 3 or more postings moves into the plan (section 12)

Roadmap at a glance

The Firmware Lab fills the last two weeks of Projects 1 and 2

26 weeks. Bars are spans; diamonds are the four gates (highlighted) and the week 8 enquiries.



26-week timeline · 3 projects, 4 gates, 2 lab blocks

Weeks 1–10 build the skills, the three projects run weeks 11–25, and each gate asks whether you still want to continue. The two Firmware Lab blocks sit in the last two weeks of Projects 1 and 2, while the hardware is soaking outdoors; the C and math threads run alongside the weekly topics.

Phase A: Foundations (weeks 1–6, 26 Oct – 6 Dec)

By week 6 you can design, solder, program and log a two-sensor circuit, and explain every part of it.

Week 1 (26 Oct): Electricity basics

- [ ] Solve Ohm's law for voltage, current or resistance without notes
- [ ] Work and measure a series circuit and a parallel circuit with a multimeter: volts, amps in series, ohms, continuity, diode test
- [ ] Explain breadboard internals; read resistor colour codes
- [ ] Calculate, build and measure an LED circuit on a 9 V battery
- [ ] Draw schematics of circuits you built, from memory

- [ ] Print the roadmap, highlight the Required items you can already do, and tick them in section 11
- [ ] Start the job-posting log: a sheet with role, link, skills named, certificates, pay, firmware-heavy or field-heavy
- [ ] Math (1 h): algebra, units and SI prefixes; rearrange V = IR and P = VI; k, m, µ
- [ ] Cold question: what resistor does a red LED need on 9 V, and why?

Covers: principles of electric circuits, electronics fundamentals, multimeter, breadboarding, basic math.

Week 2 (2 Nov): Arduino basics

- [ ] Install the Arduino IDE; upload Blink; change the timing
- [ ] A button toggles an LED using a state variable
- [ ] Blink without delay() using millis(), so the board can do two things at once
- [ ] Fade an LED with PWM on a PWM pin
- [ ] Print values with the Serial Monitor
- [ ] C thread (2 h): compile and run hello world from the Terminal; variables, types, loops, functions
- [ ] Math (1 h): functions and graphs; duty cycle as a function; y = mx + b as a calibration line
- [ ] Cold question: how do digitalWrite() and analogWrite() differ, and why only some pins? Why is delay() a problem when a device must do two jobs?

Covers: GPIO, PWM, state machines (start), C (start).

Week 3 (9 Nov): Sensors

- [ ] Explain ADC: 10 bits, 0–1023, mapping to voltage; explain what a DAC does in a sentence
- [ ] Build and calculate a voltage divider
- [ ] Read a thermistor and convert to °C
- [ ] Read a second sensor (LDR) in the same loop
- [ ] Find pinout, voltage and range in a real datasheet
- [ ] Digital design (about 2 h): truth tables for AND, OR, NOT and XOR; binary and hex; bit masks and shifts in code
- [ ] C thread (2 h): integer sizes (uint8_t, uint16_t), operators, bit manipulation
- [ ] Math (1 h): exponentials and logs; why the thermistor's curve needs ln
- [ ] Cold question: if analogRead() returns 512, what voltage is on the pin? What is 0xA5 in binary, and how do you read bit 3?

Covers: ADC/DAC, electronics fundamentals, digital design, sensors, C.

Week 4 (16 Nov): Serial communication and data logging

- [ ] Explain UART, I2C and SPI in a sentence each
- [ ] Python reads the Arduino over pyserial
- [ ] Timestamped readings append to a CSV file
- [ ] Plot the CSV with matplotlib
- [ ] Push code and a README to GitHub
- [ ] C thread (2 h): pointers, arrays, strings and structs; compile a small C program with a Makefile
- [ ] Math (1 h): rate of change and the idea of a limit
- [ ] Cold question: what port name does your Mac use, and how do you find it? What does a pointer hold?

Covers: UART, Python, Git, C, compilers/GCC, Make, algorithms and data structures (start).

Week 5 (23 Nov): Soldering

- [ ] Make clean joints; spot a cold joint by sight
- [ ] Desolder a component with wick or a pump
- [ ] Add strain relief to every wire
- [ ] Move your week 3 circuit onto perfboard
- [ ] Check continuity on every joint before powering on
- [ ] C thread (2 h): stack vs heap, static allocation, why embedded code avoids malloc
- [ ] Math (1 h): limits and continuity
- [ ] Cold question: why heat the pad rather than the solder? What lives on the stack and what on the heap?

Covers: soldering/rework, memory management, multimeter.

Week 6 (30 Nov): Consolidation build

- [ ] Standalone two-sensor logger on perfboard, logging to CSV
- [ ] Rewrite the logger firmware as a state machine (idle, sampling, logging, fault) with a ring buffer of readings
- [ ] 3D-printed enclosure made at the makerspace
- [ ] Oscilloscope session at the makerspace or borrowed: view the PWM signal and the thermistor divider; measure frequency and duty cycle

- [ ] README, wiring diagram and photos on GitHub
- [ ] Landscape tasters (2 × 45 min): CAN bus and USB; one sentence on which jobs use each
- [ ] Math (1 h): the derivative as a slope
- [ ] Cold question: what happens if you remove each component?
- [ ] Gate 1 completed (see Week 26 and review gates)

Covers: state machines, algorithms and data structures, oscilloscope, breadboarding; tasters: CAN, USB.

Phase B: Raspberry Pi and data (weeks 7–10, 7 Dec – 3 Jan)

By week 10 a Pi reads I2C and SPI sensors, stores the data in a database, serves a live dashboard, and your code has tests that run automatically.

Week 7 (7 Dec): Pi setup and Linux

- [ ] Flash Raspberry Pi OS Lite and connect over SSH with no monitor
- [ ] Use ls, cd, nano, chmod and sudo without looking them up
- [ ] Create a Python virtual environment for the project
- [ ] Blink an LED from Python through GPIO
- [ ] Run the script at boot with a systemd service; confirm after a power cycle
- [ ] Write a Bash script that rotates the log files; run it from a Makefile target
- [ ] Explain processes, threads and what systemd does
- [ ] C thread (2 h): split a program into .h and .c files; static, const and volatile
- [ ] Math (1 h): derivative rules (power, product)
- [ ] Cold question: how do you reach the Pi when you don’t know its IP address?

Covers: Bash scripting, Make, OS fundamentals, GPIO, Python, C.

Week 8 (14 Dec): I2C and SPI sensors

- [ ] Explain SDA, SCL, addresses and pull-up resistors
- [ ] Read a BME280; find it with i2cdetect
- [ ] Add a second I2C device and log both with device IDs
- [ ] Read a sensor over SPI as well (a BME280 board that exposes SPI pins, or any SPI sensor; check the pinout before you buy); compare the I2C and SPI wiring
- [ ] Capture an I2C transaction and an SPI transaction with a cheap logic analyser
- [ ] Send the two education enquiry emails (see the Education options tracker); in the Camosun email, ask about math prerequisites

- [ ] Skim the Camosun course lists against the skills map
- [ ] C thread (2 h): function pointers and callbacks; the opaque-pointer pattern for a sensor driver
- [ ] Math (1 h): the chain rule; the slope of the thermistor curve
- [ ] Cold question: what happens if two devices share an address? Why does SPI need more wires but no addresses?

Covers: I2C, SPI, logic analyser, design patterns, C.

Weeks 9–10 (21 Dec – 3 Jan): Storage, dashboard, MQTT and tests

These weeks are software-only, so they flex around the holidays. Roughly 40 hours; take the core items first.

- [ ] Move logging from CSV to SQLite; write SELECT queries over your own data
- [ ] A Flask endpoint returns the last 24 hours as JSON
- [ ] A Chart.js page graphs the data; design it deliberately
- [ ] Install Mosquitto; publish and subscribe one reading over MQTT
- [ ] Write pytest tests for your CSV parser and one SQLite query function; write one test before the code it checks
- [ ] GitHub Actions runs the tests on every push; show the badge in the README
- [ ] Work on a branch, open a pull request in your own repo, review it and merge it
- [ ] Simulate an ESP32 plus sensor sketch in Wokwi (check which sensors it supports first)
- [ ] Set up a kanban board (GitHub Projects) for the three projects, one card per week
- [ ] Explain why flash memory needs wear levelling, and what a CSV file does if power drops mid-write
- [ ] C thread (2 h each week): a linked list, a stack and a queue in C, with simple assert-based tests
- [ ] Math (1 h each week): week 9, integrals as area and accumulation; week 10, numerical integration in Python (trapezoid rule on your own log)
- [ ] Cold question: when would you choose SQLite over a CSV file, and why?

Covers: MQTT, TDD and unit testing, CI/CD, Git, hardware simulation, Agile, memory technologies and file systems, algorithms and data structures, Python.

Project 1: Solar-powered sensor node (renewable energy, weeks 11–14, 4–31 Jan)

Build a small solar-powered node that measures its own energy harvest and use, then judge whether it could run unattended through a Victoria winter.

Design. An ESP32 board reads a BME280, measures panel and battery voltage and current with an INA219 or INA226 board, sleeps between samples, and publishes readings to your Pi over MQTT. The ESP32 works in the Arduino IDE, so your week 2 skills carry over, and it runs FreeRTOS underneath. Budget roughly CAD 60–100 (approximate): a 5–10 W panel, a charge controller matched to your battery chemistry, one protected cell, the ESP32 and the sensor boards. Check each part's ratings before buying.

Suggested split. Week 11: safety, power budget, ESP32 setup, FreeRTOS basics. Week 12: breadboard build, 3-day outdoor panel log, order the Nucleo board. Week 13: perfboard, deep sleep and interrupts, enclosure, start the 7-day outdoor run, Firmware Lab Part 1. Week 14: finish the run, deliverables, KiCad schematic, Gate 2.

Safety first

- [ ] Use a cell with a built-in protection circuit and a charger matched to its chemistry
- [ ] Fuse the battery lead; never short or puncture a cell
- [ ] Charge and test in a fire-safe container; never leave it charging unattended

Learn

- [ ] Set up the ESP32 in the Arduino IDE and blink an LED
- [ ] Write a power budget: daily energy use in watt-hours
- [ ] Compare Li-ion and LiFePO4 cells for outdoor use
- [ ] Explain MPPT versus a simple charge controller
- [ ] Use deep sleep, timers and a watchdog on the ESP32
- [ ] Explain interrupts and ISR rules: set a flag, do nothing else
- [ ] Run two FreeRTOS tasks (sample, publish) joined by a queue
- [ ] Math (1 h a week): week 11, compute watt-hours by integrating a power log; week 12, capacitor charging as dV/dt = (Vs − V) / RC; week 13, the RC time constant on a breadboard; week 14, a low-pass filter and moving average in code

Build

- [ ] Log panel voltage and current outdoors for 3 days
- [ ] The node samples every 5 minutes and sleeps between readings
- [ ] Wake from deep sleep on a button interrupt as well as the timer
- [ ] Log free heap memory with each sample
- [ ] Solder the circuit onto perfboard
- [ ] Weatherproof the enclosure for an outdoor test
- [ ] Run outdoors for 7 days; record energy in and out

Firmware Lab Part 1 (weeks 13–14, about 11 hours, during the outdoor run)

Order the Nucleo board in week 12 (roughly CAD 25–40, approximate; its built-in ST-LINK debugger means no extra probe).

- [ ] Install arm-none-eabi-gcc, CMake and OpenOCD on your Mac
- [ ] Blink the Nucleo from a CMake build, flash it, and attach GDB over SWD
- [ ] Set a breakpoint, inspect a variable, step through a function
- [ ] A button interrupt toggles an LED; the ISR only sets a flag
- [ ] A hardware timer interrupt replaces delay()
- [ ] Read the startup code: vector table, reset handler, main(); find the memory map in the reference manual
- [ ] Cold question: what happens between power-on and main()?

Deliverables

- [ ] Chart of expected versus measured energy
- [ ] Dashboard showing panel power, battery voltage and temperature
- [ ] KiCad schematic of the node as a PDF in the repo
- [ ] System diagram and one-page brief: would it survive December in Victoria?
- [ ] GitHub repo with README, parts list and photos
- [ ] Landscape tasters (2 × 45 min): Modbus/RS-485 and Ethernet; one sentence on which jobs use each
- [ ] Explain three differences between C and C++ (the Arduino and ESP32 code you wrote is C++)
- [ ] Cold question: how many cloudy days can the node survive, and how did you calculate it?
- [ ] Gate 2 completed (see Week 26 and review gates)

Covers: power management, watchdog, timers/counters, interrupts, RTOS basics, FreeRTOS, JTAG/SWD, GDB, compilers/GCC, Make/CMake, GPIO, computer architecture, memory management, hardware design basics, C++ (explain), basic math; tasters: Modbus, Ethernet.

Project 2: Water-quality logger (marine conservation, weeks 15–20, 1 Feb – 14 Mar)

Build a waterproof logger that records temperature, turbidity and conductivity at a real site, then test its data against a public reference.

Design. Reuse the Project 1 power design. A DS18B20 waterproof probe, a turbidity sensor and a conductivity probe feed an ESP32 or your Pi. Check each probe's range first: many hobby probes saturate in seawater, so you may have to treat readings as relative and say so in your report.

Suggested split. Week 15: learn the sealing topics, talk to a conservation group, pick a site and get permission. Week 16: Fusion 360 and the first print. Week 17: second print, 24-hour leak test, glands and conformal coating. Week 18: temperature calibration and bench test. Week 19: deploy for 7 days or more, Firmware Lab Part 2. Week 20: retrieve, inspect, compare with the public station, write the report, choose node 3, Gate 3.

Learn

- [ ] Read IP ratings and what each level requires
- [ ] Compare gaskets, potting resin and conformal coating, and when to use each
- [ ] Explain cable glands and how wires leave a sealed enclosure
- [ ] Understand corrosion, biofouling and sensor drift
- [ ] Learn enough Fusion 360 to model a two-part box with a gasket groove
- [ ] Store timestamps in UTC; flag gaps and outliers in the data
- [ ] Math (1 h a week): weeks 15–16, exponential decay and time constants in sensor settling; weeks 17–18, PID intuition (proportional is the present, integral the past, derivative the future); weeks 19–20, sampling and aliasing (what a 5-minute sample interval can and can't see)

Build

- [ ] Print the enclosure in PETG or ASA rather than PLA; iterate the fit at least once
- [ ] Leak-test the empty enclosure for 24 hours with tissue inside
- [ ] Install cable glands and apply conformal coating to the board
- [ ] Calibrate temperature against a reference thermometer in ice and warm baths; record the offset
- [ ] Talk to one local conservation or stewardship group about a measurement they would use
- [ ] Get permission and choose a safe site (dock, creek or tank)
- [ ] Deploy for 7 days or more; retrieve and inspect for fouling

Firmware Lab Part 2 (weeks 19–20, about 11 hours, during the deployment)

- [ ] Read an ADC channel into a buffer using DMA
- [ ] Transmit over UART using DMA

- [ ] Find the clock tree in the reference manual; change the system clock and measure the effect
- [ ] Move your week 3 thermistor conversion into a pure C function and test it on your Mac
- [ ] Cold question (lab): why use DMA instead of polling, and what does the CPU do meanwhile?

If Gate 2 said Adjust, drop this block first.

Deliverables

- [ ] Compare your temperature trend with the nearest public ONC or government station; compare trends, not values
- [ ] Two-page field report: method, data figure, limits, what you would change
- [ ] GitHub repo with README, enclosure files and photos
- [ ] Choose node 3 for Project 3 (indoor humidity and air quality, plant watering, or a similar low-voltage build)
- [ ] Landscape tasters (2 × 45 min): Zephyr and LoRa/LoRaWAN; one sentence on which jobs use each
- [ ] Cold question: what would make a scientist distrust your data, and how did you test for it?
- [ ] Gate 3 completed (see Week 26 and review gates)

Covers: DMA, clock management, ADC, UART, TDD and unit testing, sensors and actuators, soldering, hardware design basics, basic math (RC, PID, sampling); tasters: Zephyr, LoRa.

Project 3: Multi-node monitoring system (practical IoT, weeks 21–25, 15 Mar – 18 Apr)

Build a three-node system with a Pi hub, a dashboard and alerts, designed like a product. This is where your architecture and UX experience set you apart.

Design. Nodes 1 and 2 are your solar and water nodes. Node 3 is something you will actually use, chosen by week 20: indoor humidity and air quality, plant watering, or a similar low-voltage build. Two variants fit the same system: an accessible control or feedback interface for assistive technology (a small display or indicator driven by a state machine), or your bloom-tracker display driven by a data feed. Never wire mains voltage; use certified plug-in meters if you want power data.

Suggested split. Week 21: architecture, requirements and test plan, wireframes, order node 3 parts. Week 22: broker with a password, shared JSON schema, nodes publishing, Wireshark capture. Week 23: storage, dashboard and alerts. Week 24: watchdogs, over-

the-air update, the bug write-up, CI tests, the open-source pull request. Week 25: five-minute user test, deliverables, demo rehearsal.

Architecture

- [ ] One-page diagram: nodes, MQTT broker, storage, dashboard, alerts and failure paths
- [ ] Record three decisions with reasons: protocol, storage (SQLite or InfluxDB) and security
- [ ] Failure analysis: what happens when Wi-Fi drops, the broker dies or a node reboots?
- [ ] Write 8–10 requirements, then a test plan where each test traces to a requirement

Build

- [ ] Mosquitto broker on the Pi with a username and password
- [ ] All three nodes publish JSON with a shared schema and a device ID
- [ ] Capture the MQTT traffic in Wireshark and find the CONNECT packet
- [ ] Data is stored and shown on a dashboard (Grafana, or Flask with Chart.js)
- [ ] Alerts for low battery and for no data in 30 minutes
- [ ] Watchdog on every node; over-the-air update on one
- [ ] Unit tests for the schema validation and alert logic, passing in CI
- [ ] Diagnose one real bug with a logic analyser or serial logs; write up the cause

UX

- [ ] Sketch wireframes before building
- [ ] Show units, states and stale-data warnings clearly
- [ ] Run a five-minute test with one other person and change one thing

Collaboration

- [ ] Open one small pull request on an open-source library you used (a docs fix counts) by week 24
- [ ] When stuck for 90 minutes, post a question on EE Stack Exchange or r/embedded and write up the answer

Deliverables

- [ ] GitHub repo, architecture brief and a demo you can run in an interview
- [ ] Cold question: if node 2 goes silent at 3 a.m., how will you find out, and what did you design to cope?

Covers: MQTT, Wi-Fi, TCP/IP, bootloader/DFU, watchdog, embedded security, IoT, embedded GUI, V-model, Agile, CI/CD, TDD and unit testing, state machines, teamwork, communication, problem-solving.

Week 26 and review gates

Week 26 turns three projects into a portfolio and a decision; four earlier gates check whether you still want to continue.

Week 26 (19–25 Apr): Portfolio and decision

- [ ] One portfolio page presenting all three projects with photos, diagrams and briefs
- [ ] CV rewritten to lead with the projects and frame software, architecture and UX as your edge
- [ ] Re-tick the skills map (section 11); count the 30 non-soft Required items at Use or above
- [ ] List the Required items still at Explain or Taster only, and feed them into the rubric
- [ ] Compare the roadmap, the Camosun course lists and your posting tags
- [ ] Optional taster (45 min): the first video of the DigiKey “Introduction to Embedded Linux” playlist
- [ ] One-page decision rubric: energy, evidence of fit, pay, lifestyle and education cost
- [ ] Choose one path: apply for roles now, apply for study, extend three months, or step sideways within tech

Review gates

Score each honestly, then circle a decision.

Gate	When	Questions	Decision
1. Foundations	End of week 6 (6 Dec)	Can you build, solder and log without a tutorial? Did debugging absorb or drain you? Did C and bit-level detail energise you or drain you?	Go / Adjust / Stop
2. Project 1	End of week 14 (31 Jan)	Did the power and safety thinking interest you? Did interrupts and the debugger feel like power or like a drag? Are you still doing about 20 hours a week?	Go / Adjust / Stop

Gate	When	Questions	Decision
3. Project 2	End of week 20 (14 Mar)	Did fieldwork and the conservation conversation energise you? Which roles now look best? What do your tagged postings ask for that the skills map lacks?	Go / Adjust / Stop
4. Final	End of week 26 (25 Apr)	Would you do this work daily? What does your evidence file say? Which way do postings and energy scores point: firmware, field systems, conservation or something else? Which education path, if any?	Go / Adjust / Stop

Suggested rule (edit it): two low-energy gates in a row means pause and revisit the role list rather than push harder. If your energy is high at fieldwork and low at C-level detail, that is a result, not a failure: it points toward the technician, conservation and IoT solutions roles.

Weeks to protect: weeks 6, 14, 20 and 26 each carry a gate, a taster and a decision. Don't schedule anything else heavy in them.

Roadmap skills map

Tick a box only when you can do the thing without a tutorial open. Depth levels: **Taster** (watched an overview, one sentence on what it's for), **Explain** (say it aloud without notes), **Use** (done inside a project), **Build** (shipped in a portfolio repo with a README). Your target for the 30 non-soft Required items is Use or above by week 26, apart from the few marked Explain.

Required: software (11)

- [ ] **C**: C thread weeks 2–14, then the Firmware Lab. Use
- [ ] **Algorithms and data structures**: arrays and structs (wk 4), ring buffer (wk 6), linked list, stack and queue (wks 9–10). Use
- [ ] **Design patterns**: state machine (wk 6), callbacks and opaque pointer (wk 8). Explain
- [ ] **State machines**: weeks 2 and 6, Project 3 node 3. Build
- [ ] **Memory management**: stack vs heap (wk 5), heap logging (Project 1). Use
- [ ] **JTAG / SWD**: Firmware Lab Part 1. Use
- [ ] **GDB**: Firmware Lab Part 1. Use
- [ ] **Compilers / GCC**: week 4, then arm-none-eabi-gcc in the lab. Use

- [ ] **Make / CMake**: Make in weeks 4 and 7, CMake in the lab. Use
- [ ] **Git**: week 4; branches and pull requests in weeks 9–10. Build
- [ ] **TDD and unit testing**: weeks 9–10, Project 3 CI, lab Part 2 host test. Use

Required: software ∩ hardware (12)

- [ ] **RTOS basics**: two FreeRTOS tasks and a queue (Project 1). Use
- [ ] **GPIO**: week 2. Use
- [ ] **ADC / DAC**: ADC week 3, lab Part 2; DAC explained. Use / Explain
- [ ] **Timers / counters**: Project 1, lab Part 1. Use
- [ ] **PWM**: week 2. Use
- [ ] **Watchdog**: Projects 1 and 3. Build
- [ ] **Interrupts**: Project 1, lab Part 1. Use
- [ ] **DMA**: lab Part 2. Use
- [ ] **Clock management**: lab Part 2. Use
- [ ] **UART**: week 4, lab Part 2. Use
- [ ] **I2C**: week 8. Build
- [ ] **SPI**: week 8. Use

Required: hardware (7)

- [ ] **Basic math and calculus**: math thread, weeks 1–20 (algebra to PID intuition). Use
- [ ] **Principles of electric circuits**: week 1. Use
- [ ] **Electronics fundamentals**: weeks 1–3, Project 1 power design. Use
- [ ] **Digital design**: week 3. Explain
- [ ] **Computer architecture**: Firmware Lab Part 1 (memory map, reset sequence). Explain
- [ ] **Multimeter**: week 1. Use
- [ ] **Logic / protocol analyser**: week 8, Project 3 bug. Use

Required: soft skills (6)

- [ ] **Communication**: READMEs, briefs, the interview demo; explain one project to a non-technical friend
- [ ] **Problem-solving and critical thinking**: cold questions; the Project 3 bug write-up
- [ ] **Teamwork and collaboration**: conservation-group conversation (Project 2); one open-source pull request (Project 3)
- [ ] **Organisation and time management**: fit journal hours against the 20-hour plan
- [ ] **Self-driven and independent**: gates; public questions when stuck for 90 minutes

- [ ] **Adaptability and patience**: log one debugging session over 2 hours and what you tried

Recommended, in the core plan

- [ ] **Python** (weeks 4–10, Projects 1–3), **Bash scripting** (wk 7), **Breadboarding** (wk 1), **Soldering / rework** (wk 5). Use
- [ ] **Oscilloscope**: week 6. Use
- [ ] **Power management**: Project 1 deep sleep. Use
- [ ] **Bootloader / DFU**: over-the-air update, Project 3. Use
- [ ] **Wi-Fi** and **MQTT**: weeks 9–10, Projects 1–3. Use
- [ ] **FreeRTOS**: Project 1. Use
- [ ] **Hardware simulation**: Wokwi, weeks 9–10. Use
- [ ] **CI/CD**: weeks 9–10, Project 3. Use
- [ ] **Agile / Scrum**: kanban board from week 9. Explain
- [ ] **V-model**: Project 3 requirements and test plan. Explain
- [ ] **OS fundamentals**: week 7. Explain
- [ ] **C++**: three differences from C, Project 1. Explain
- [ ] **Memory technologies and file systems**: week 10. Explain
- [ ] **Hardware design basics**: KiCad schematic, Project 1. Use

Recommended and Optional, as tasters (week of the gate)

- [ ] **CAN** and **USB**: week 6. Taster
- [ ] **Modbus / RS-485** and **Ethernet**: week 14. Taster
- [ ] **Zephyr** and **LoRa**: week 20. Taster
- [ ] **Embedded Linux** (kernel, drivers, Yocto and Buildroot): week 26, optional. Taster

Optional, picked on purpose

- [ ] **IoT** (specialization): Projects 1–3
- [ ] **Embedded security**: Project 3 decision record
- [ ] **Embedded GUI**: the Project 3 UX variant
- [ ] **Sensors and actuators**: sensors throughout; one actuator if node 3 is plant watering
- [ ] **TCP/IP**: Wireshark capture, Project 3

Stretch menu and parked list

Use the stretch menu only when you are ahead of the plan and your last gate scored Go. The items point in different directions on purpose, so you can sample without committing.

Stretch menu

- [ ] **PCB design / EMC (Recommended):** lay out the Project 1 node in KiCad and order a board; check lead time and shipping to Canada, and order by week 21 if you want it for Project 3
- [ ] **Bluetooth (Recommended):** read a sensor value from your phone over BLE with a free scanner app; any week after week 14
- [ ] **Vendor SDK:** rewrite one Project 1 module in ESP-IDF (which uses CMake), as the roadmap suggests professionals do
- [ ] **RTOS on the Nucleo:** run a FreeRTOS example through STM32Cube and compare it with the ESP32 version
- [ ] **Edge AI taster (Optional):** watch one overview of running a small model on a microcontroller and note what sensor data it would need

Parked list

These are deliberately out of the 26 weeks. Move one in only if it appears in 3 or more of your tagged postings.

- C++ in depth, Assembly, Rust, Docker, SVN, SIL/HIL testing, standards and certifications
- Linux kernel, device drivers, U-Boot, Buildroot and Yocto (week 26 taster only)
- QNX, µC/OS, RT-Thread and other RTOSes
- PCIe, Zigbee, Thread, Matter, UWB, Profinet, EtherCAT, cellular (GSM, LTE, LTE-M, NB-IoT), LIN, MOST, FlexRay, MIPI
- DSP, control theory beyond the PID intuition in the math thread, FPGA, AUTOSAR

Education options tracker

Several intakes close before this curriculum ends, so send enquiries in week 8 rather than waiting for the final gate. Details come from search results; confirm each with the provider.

Option	Where	Length and start	What to ask or check	Done
Camosun Electronics and Computer Engineering Technology (Renewable Energy)	Victoria	2.5 years; September start	Entry requirements and math prerequisites, September 2027 deadline, study permit route	☐

Option	Where	Length and start	What to ask or check	Done
Camosun Electronics and Computer Engineering Access certificate	Victoria	8 months	Whether you need it; its reserved seat in the diploma	☐
Camosun Electrical Engineering Technology (Marine and Industrial)	Victoria	2.5 years; September start	Shares year 1 with the diploma above; marine electrical focus; math prerequisites	☐
RMIT Advanced Diploma of Electronics and Communications Engineering	Melbourne	2 years; February intake listed, confirm	Credit for your BDes and software career; 2027 domestic fee; intake dates	☐
UniMelb Graduate Certificate in Environment	Melbourne	6 months full-time; March or July	Builds environment knowledge, not hardware skills; confirm fees (sources differ)	☐
UVic GIS Skills and Mapping, Mapping with Drones	Online	Short courses; start varies	Next intake date (a January run is likely, unconfirmed)	☐
Marine safety tickets (SDV-BS, SVOP, marine first aid)	Online and local	Days	Which providers run in Victoria; whether target employers ask for them	☐

- [ ] Week 8: email Camosun's department chair and Camosun International about entry, math prerequisites, deadlines and permits
- [ ] Week 8: email RMIT admissions about credit for your degree and work experience
- [ ] Week 8: compare the Camosun course lists with the skills map and note the gaps
- [ ] Week 12: book one short course (a marine ticket or GIS) to finish before week 20
- [ ] Week 26: replace the questions above with answers and rank the options

Weekly fit journal

Spend two minutes on this each Sunday. Energy 1 means drained and 5 means you could keep going; the pattern across weeks matters more than any single score. Compare the Hours column with the 20-hour plan.

Week	Hours	Energy (1–5)	Most absorbing task	Most draining task	Would I do this daily?
1 (26 Oct)					
2 (2 Nov)					
3 (9 Nov)					
4 (16 Nov)					
5 (23 Nov)					
6 (30 Nov)					
7 (7 Dec)					
8 (14 Dec)					
9 (21 Dec)					
10 (28 Dec)					
11 (4 Jan)					
12 (11 Jan)					
13 (18 Jan)					
14 (25 Jan)					
15 (1 Feb)					
16 (8 Feb)					
17 (15 Feb)					
18 (22 Feb)					
19 (1 Mar)					
20 (8 Mar)					
21 (15 Mar)					
22 (22 Mar)					
23 (29 Mar)					
24 (5 Apr)					
25 (12 Apr)					
26 (19 Apr)					

Resources, purchases and sources

The first group are the sites and series you have said work for you; the second group comes from the roadmap's own resource lists.

Your resources

- CircuitBread: theory at builder-friendly depth
- All About Circuits textbook: fills gaps when a topic doesn't click
- Paul McWhorter's Arduino and Raspberry Pi video series on YouTube: methodical, good for first builds
- SparkFun Learn, Adafruit Learn and Random Nerd Tutorials: sensor and ESP32 guides
- Raspberry Pi documentation and Arduino docs

From the roadmap's lists

- C: Microchip University "Syntax and Structure of C"; C by Example
- State machines and patterns: *Making Embedded Systems* (Elecia White); the Quantum Leaps state-machine and event-driven playlists
- Debugging and startup code: Memfault Interrupt blog, "A Deep Dive into ARM Cortex-M Debug Interfaces", "Advanced GDB Usage" and "From Zero to main()"
- STM32: DigiKey "Getting Started with STM32 and Nucleo"; Phil's Lab
- RTOS: DigiKey "Introduction to RTOS"; *Mastering the FreeRTOS Real Time Kernel*
- Digital design: Tiny Tapeout digital design guide
- Math: Khan Academy Calculus 1; Brian Douglas's control systems lectures for PID intuition
- Simulation: Wokwi
- Tasters: Microchip University "Introduction to USB 2.0"; "CAN Bus: A Beginners Guide"; "What is Modbus and How does it Work?"; "What is RS-485?"; Microchip University "Ethernet Fundamentals"; DigiKey "Introduction to Zephyr"; "LoRa / LoRaWAN De-Mystified"
- Roadmap and full resource lists: github.com/m3y54m/Embedded-Engineering-Roadmap (Embedded Systems Engineering Roadmap by Meysam Parvizi, CC BY-SA 4.0)

Purchases (approximate; check each before buying)

- [ ] Project 1 parts: roughly CAD 60–100 (panel, charge controller, protected cell, ESP32, sensor boards)
- [ ] STM32 Nucleo board: roughly CAD 25–40; order in week 12

- [ ] A sensor board that exposes SPI pins (for week 8)
- [ ] Oscilloscope: borrow from a makerspace or library for week 6 before buying

Sources: programme details in the education tracker come from search results, so confirm them directly. Parts costs are approximate. The roadmap's Required, Recommended and Optional ratings were read from its PDF, version 2.0.3.